



AVARA

WEAR IT BEFORE YOU BUY IT

OUR TEAM



NGUYỄN MINH KHÔI
ITCSIU22210

Full Stack Developer

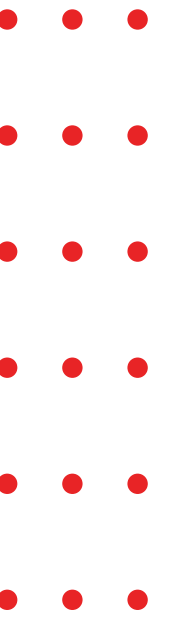


VƯƠNG QUÁN SIÊU
ITCSIU22270

Full Stack Developer

TABLE OF CONTENT

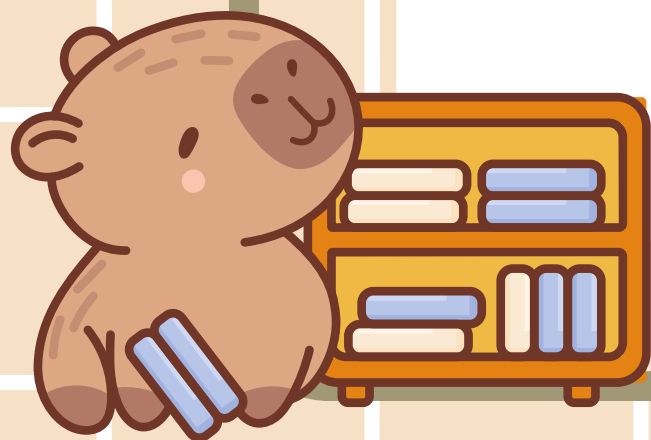
- Problem statement
- Tech Stack
- High-level Architecture
- Use Case Diagram
- Entity Relationship Diagram
- MVC Pattern
- Sequence Diagram
- Implementation
- Demo



Problem Statement

Many online shoppers struggle to determine how clothing will look and fit before purchasing. Traditional e-commerce platforms only provide static product images, leading to uncertainty, poor buying decisions, and high product return rates.

Solution: AVARA introduces an AI-powered Virtual Try-On system that allows customers to visualize garments on themselves before making a purchase.



— OUR BESTSELLERS

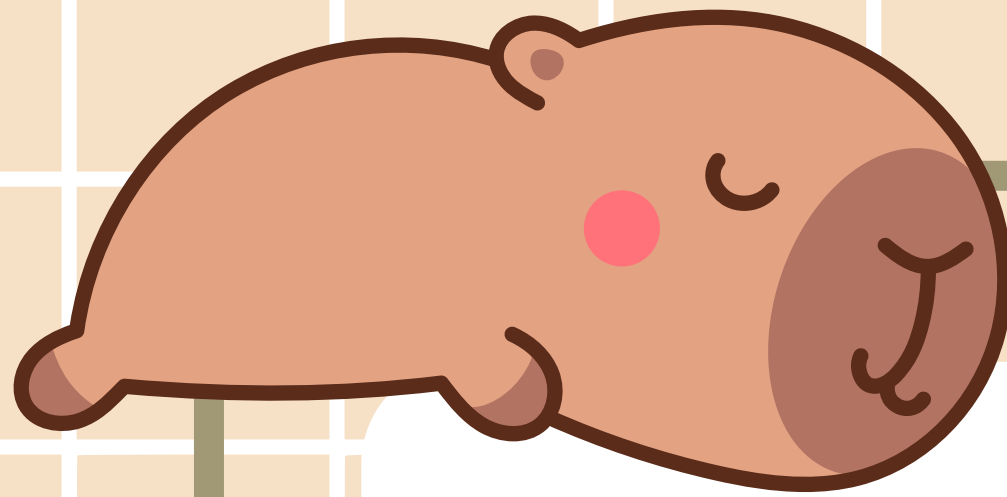
Latest Arrivals

SHOP NOW —



LATEST COLLECTION —

Explore our latest collection of fashion-forward products, carefully curated to elevate your style. From trendy apparel to must-have



Tech Stack

Frontend

- React.js
- Tailwind CSS
- Vite

Backend

- Node.js
- Express.js

Database

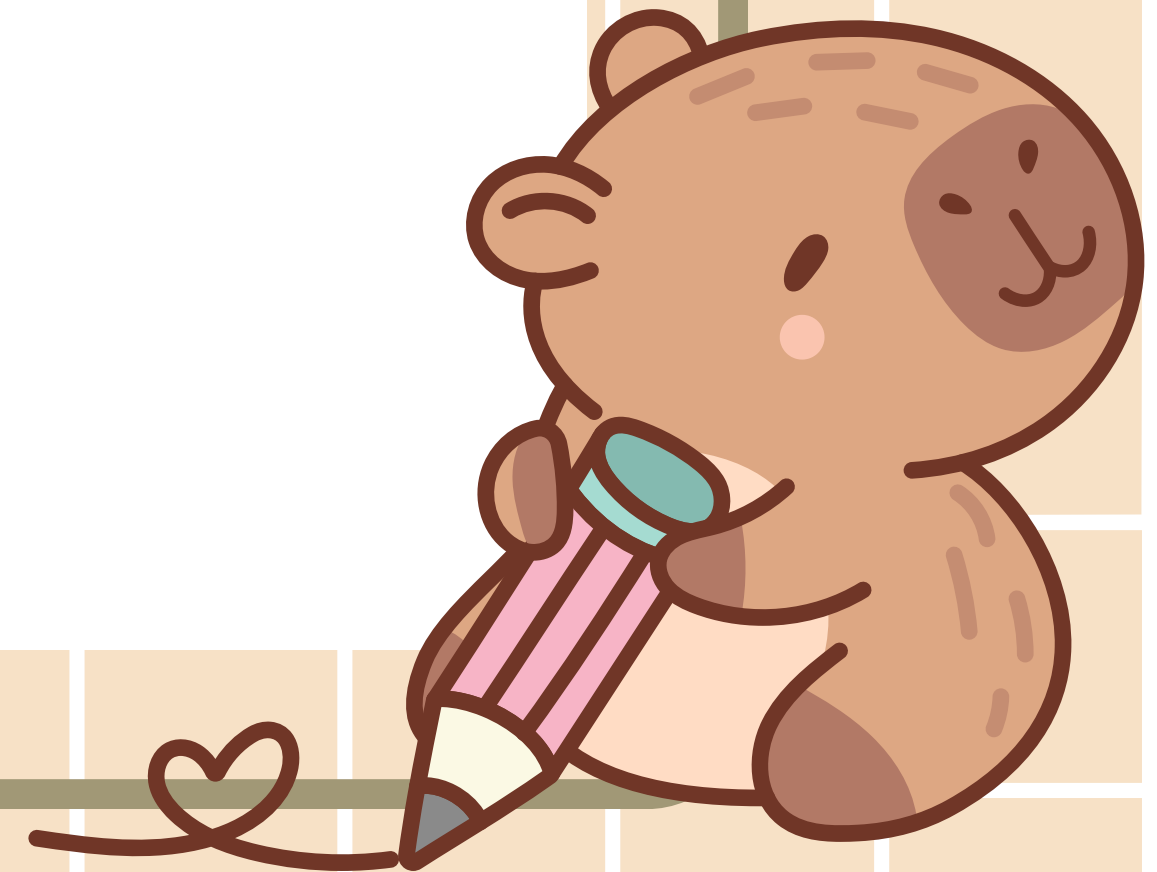
- MongoDB Atlas
- Mongoose

AI & Services

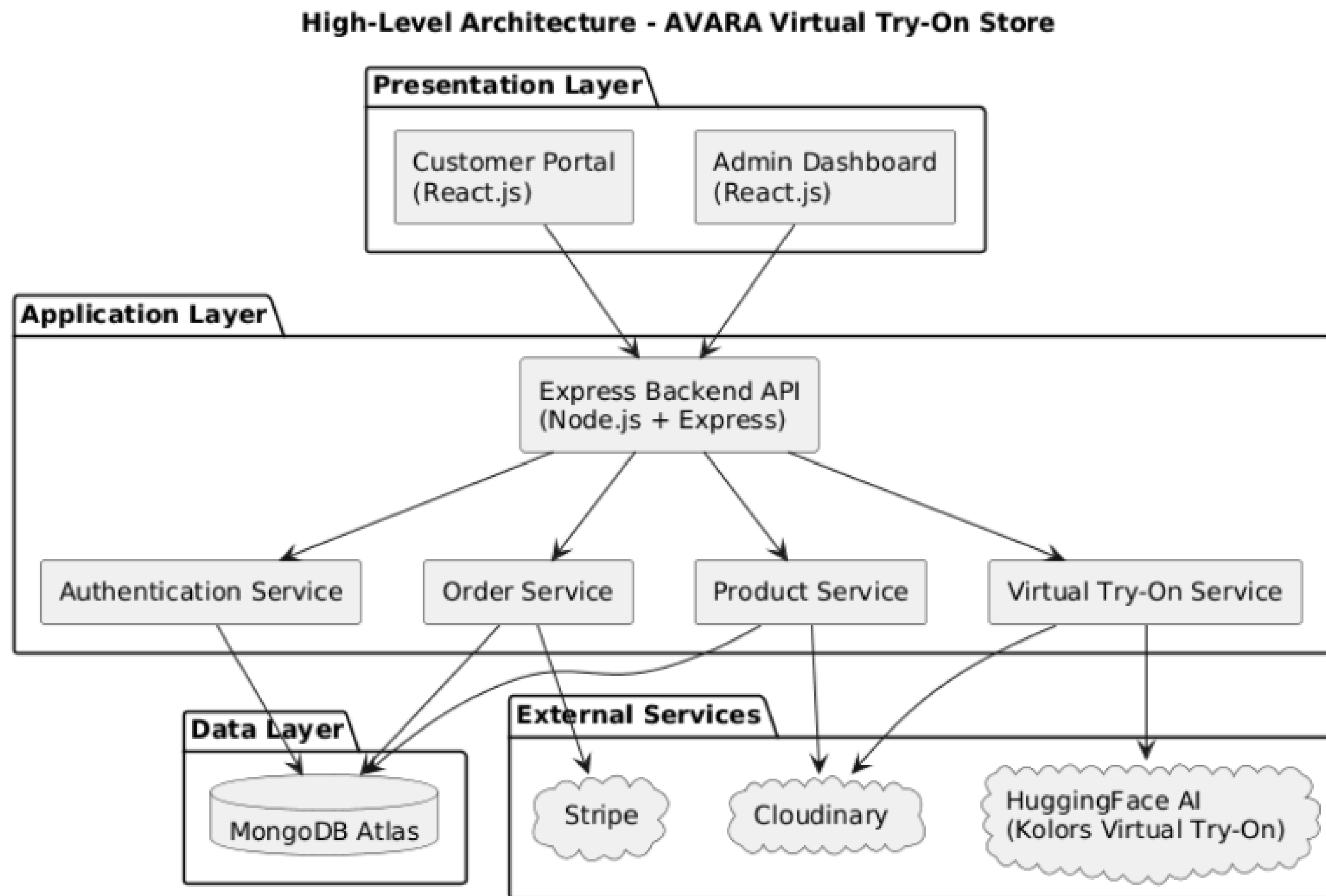
- Kolos Virtual Try-On
- Cloudinary
- Stripe

Tools

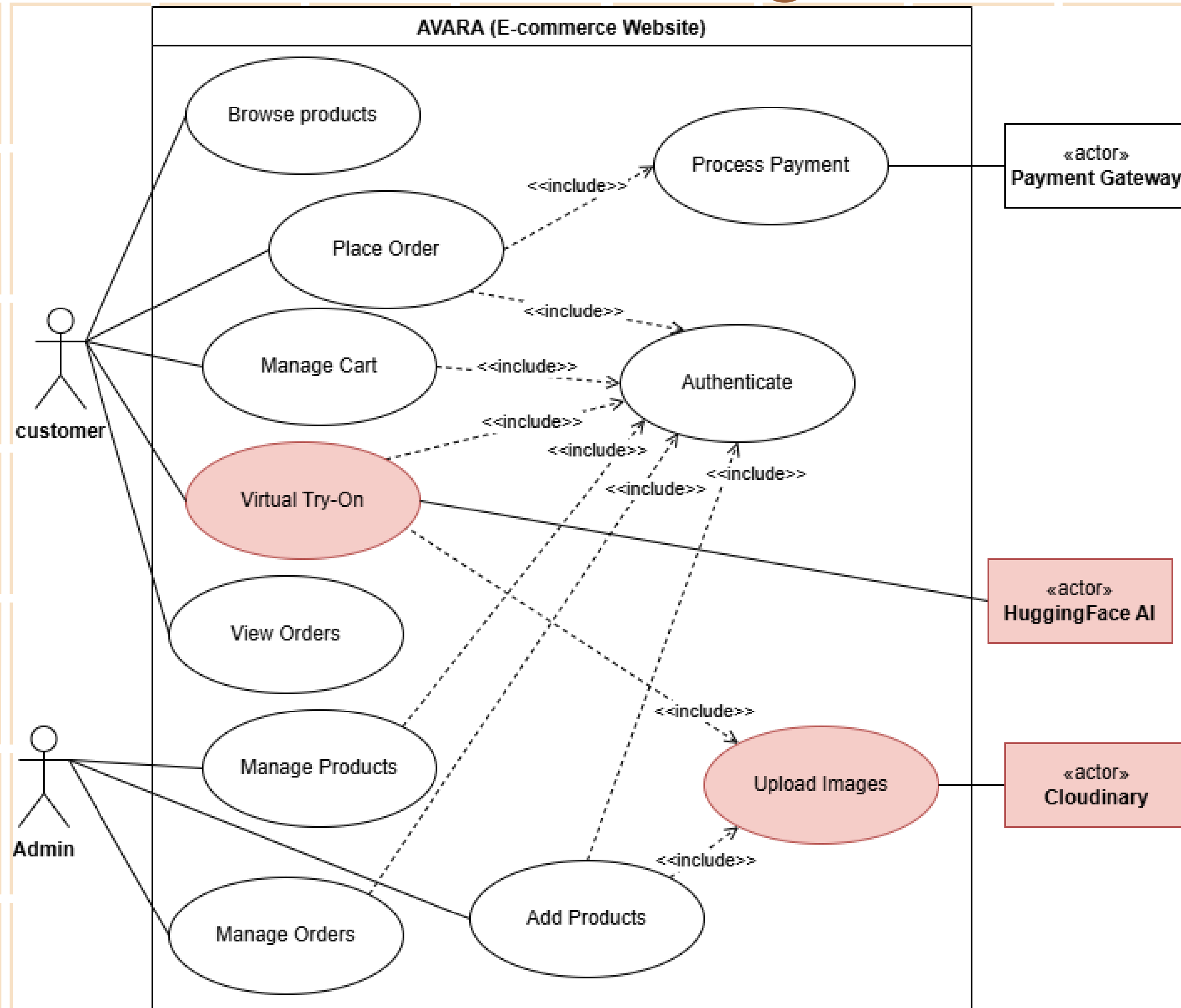
- GitHub
- Postman
- VS Code



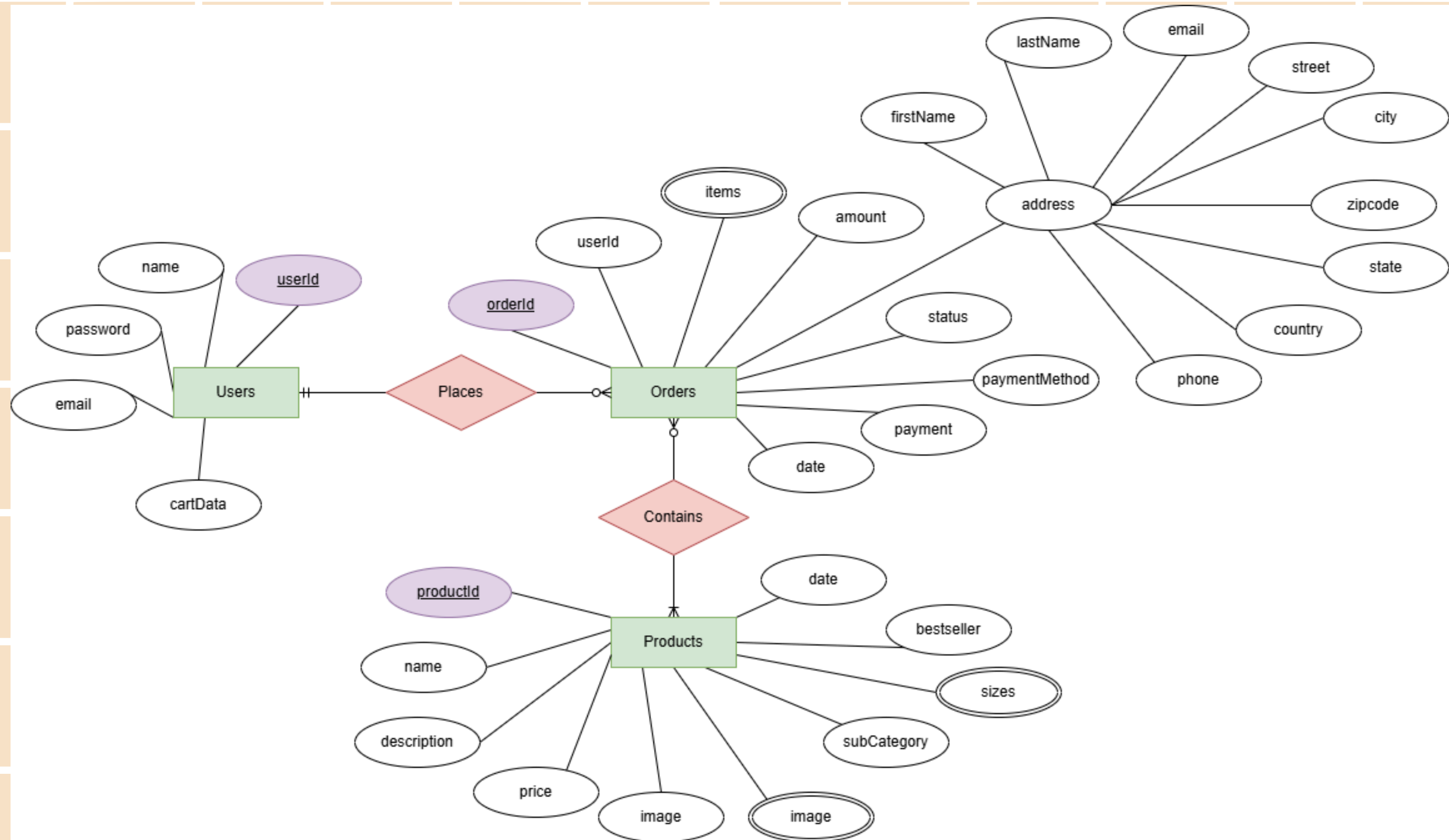
High-level Architecture



Use Case Diagram

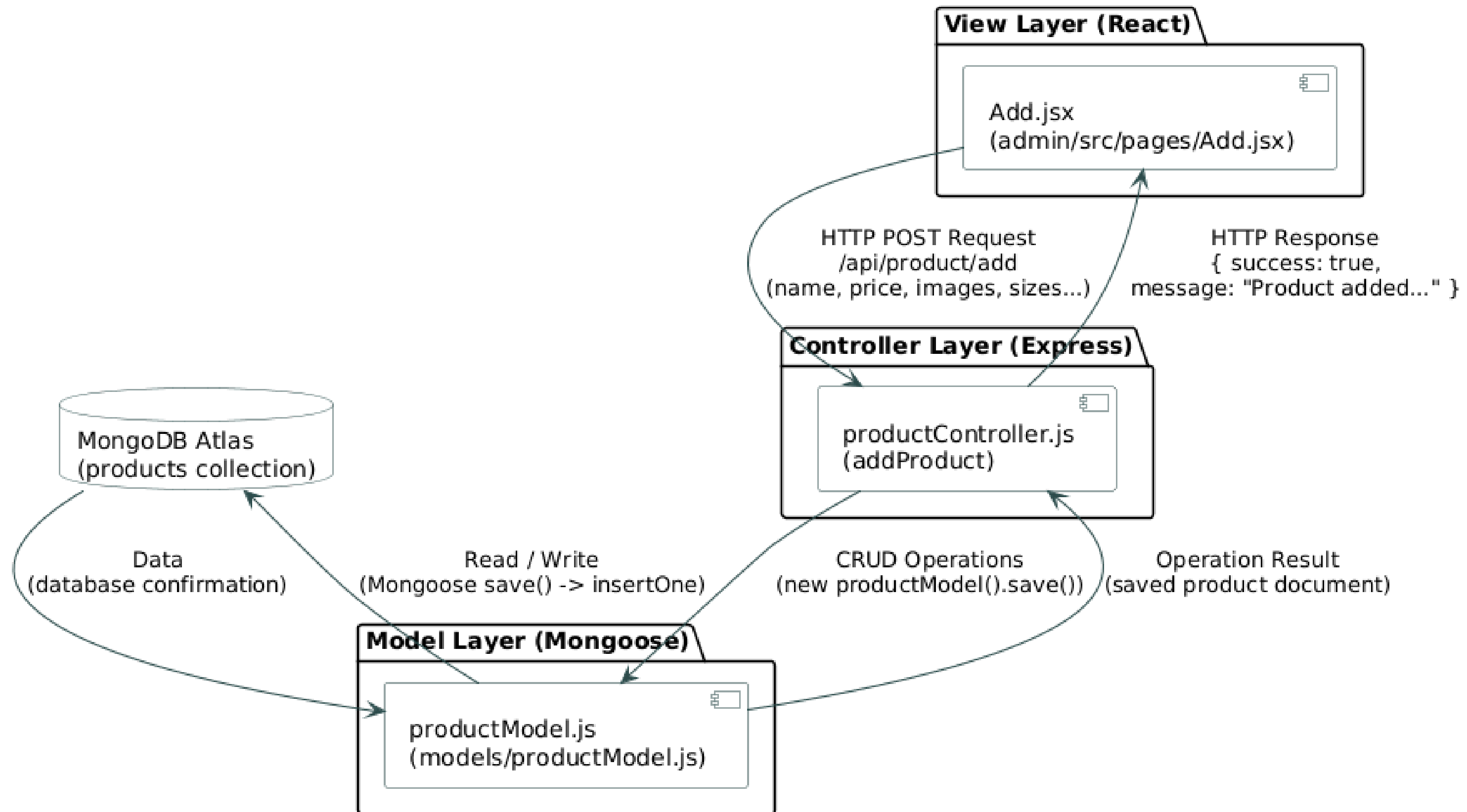


ERD

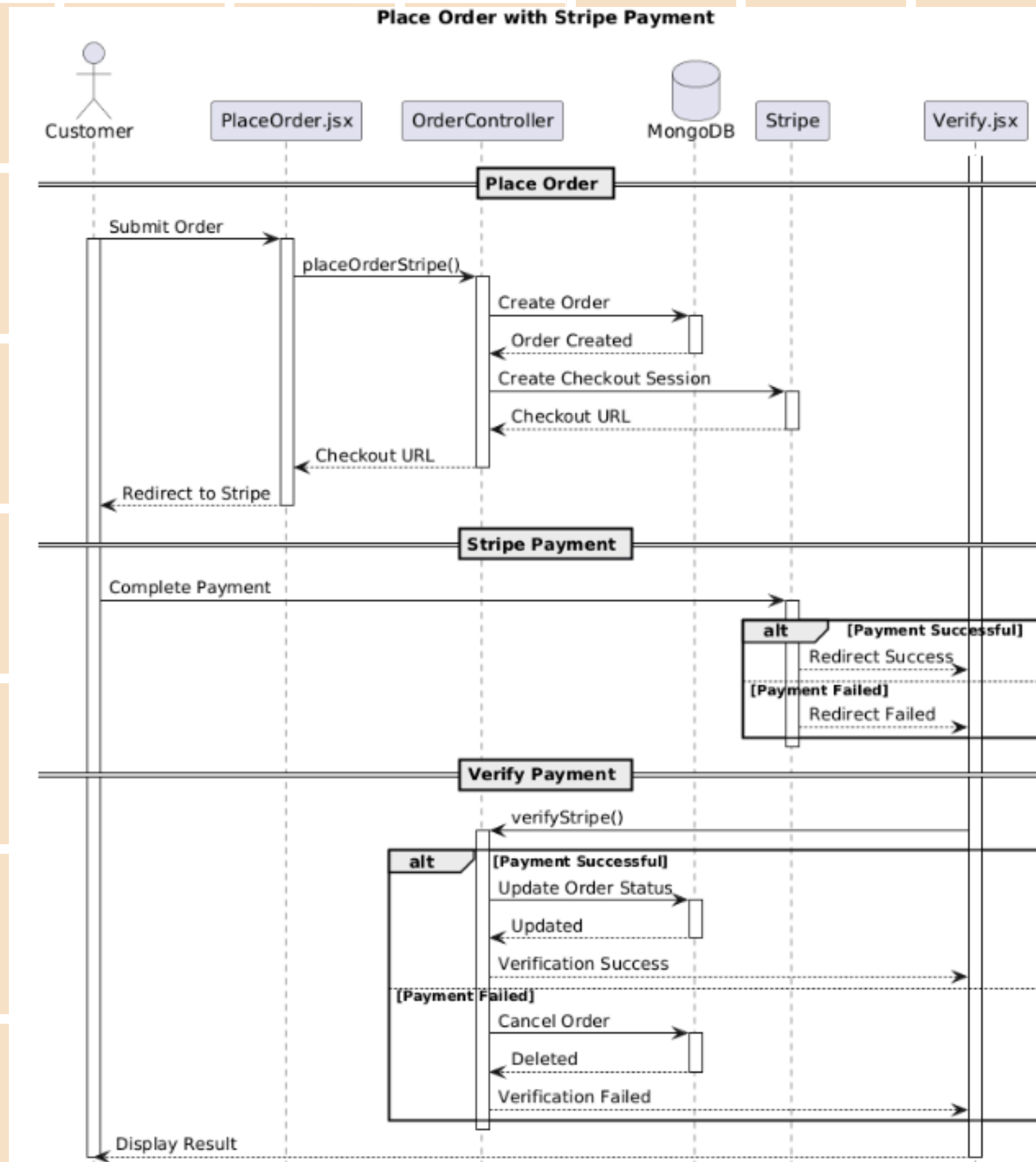


MVC Pattern

MVC Architecture — Admin Add Product Example

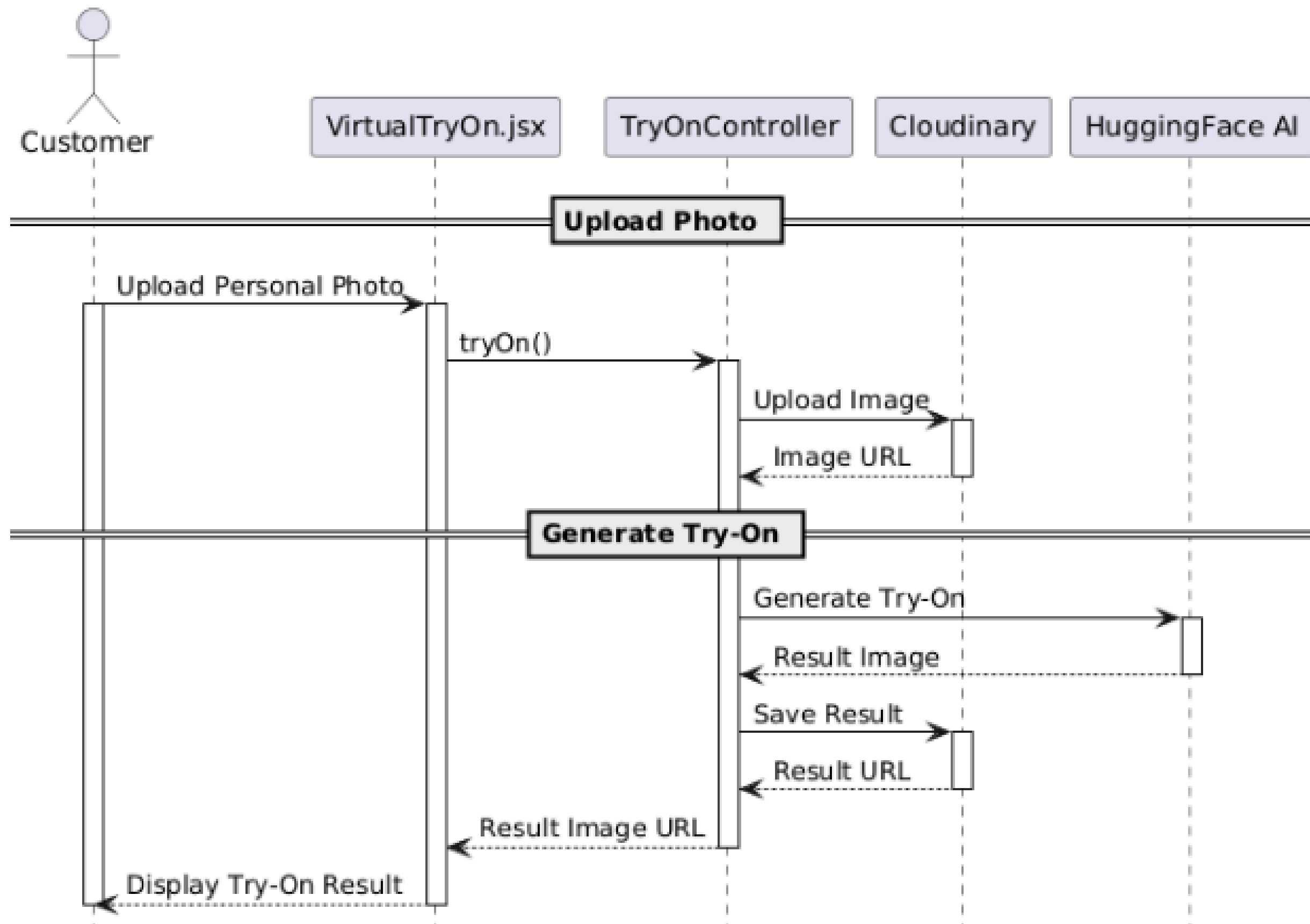


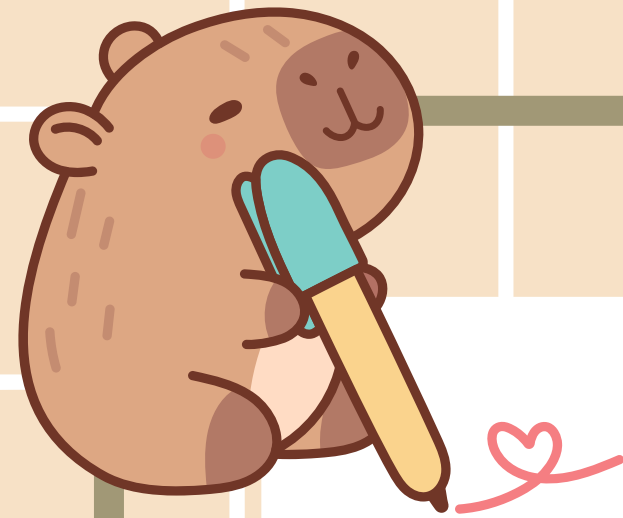
Sequence Diagram



Sequence Diagram

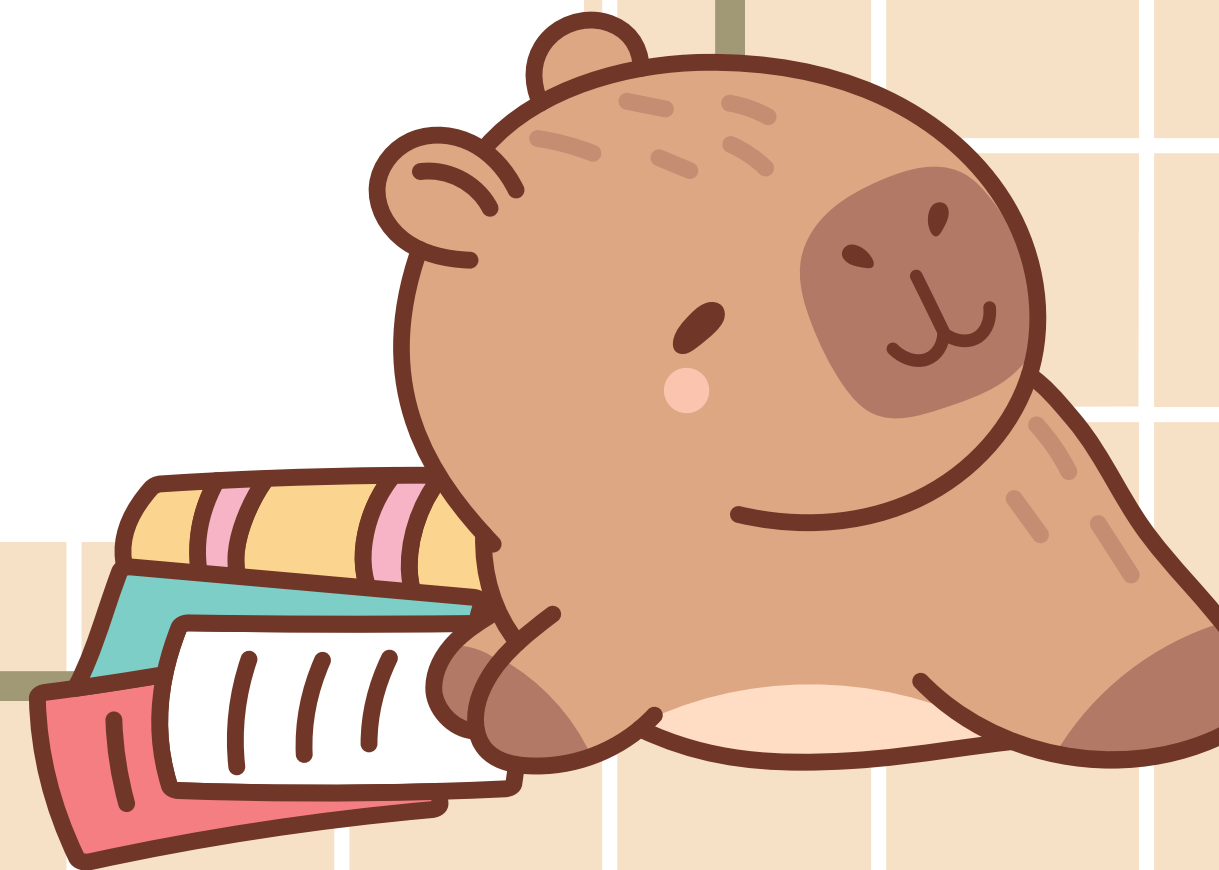
Virtual Try-On Process





Implementation

AI Virtual Try-On
feature



VIEW Layer – React Frontend

```
// Capturing user inputs and triggering the controller API request
const handleGenerate = async () => {
  setIsLoading(true);
  try {
    const formData = new FormData();
    formData.append("personImage", personImage);      // File uploaded by user
    formData.append("garmentImageUrl", garmentImage);  // Selected product image URL
    formData.append("subCategory", product.subCategory);

    const response = await axios.post(`${backendUrl}/api/tryon`, formData, {
      headers: {
        token, // JWT client authorization token
        "Content-Type": "multipart/form-data",
      },
      timeout: 300000 // 5-minute timeout for AI processing
    });

    if (response.data.success) {
      setResultImage(response.data.resultImageUrl); // Updates View with final CDN
      output
      setClothType(response.data.clothType);
    }
  } catch (error) { ... }
};
```

CONTROLLER Layer – Express Route & API Logic

```
// Express Route configuration showing middleware intercepts
tryonRouter.post(
  "/",
  authUser,          // 1. Authenticate user JWT token
  upload.single("personImage"), // 2. Parse file using Multer middleware
  tryOn              // 3. Forward to the Controller function
);
```

```
// Controller handling Hugging Face API prediction
const tryOn = async (req, res) => {
  const { garmentImageUrl, subCategory } = req.body;
  const clothType = detectClothType(subCategory);

  // 1. Connect to HuggingFace Gradio Space API
  const client = await Client.connect("Kwai-Kolors/Kolors-Virtual-Try-On", {
    hf_token: process.env.HF_TOKEN,
  });

  // 2. Execute Prediction
  const result = await client.predict(2, [
    handle_file(personImageUrl), // Stored user photo public URL
    handle_file(garmentImageUrl), // Garment product image CDN URL
    0,                          // Seed number
    true,                       // Randomize seed
  ]);

  const outputUrl = result?.data?.[0]?.url || result?.data?.[0];
  // Next: Passes buffer output to the persistent model storage layer
};
```

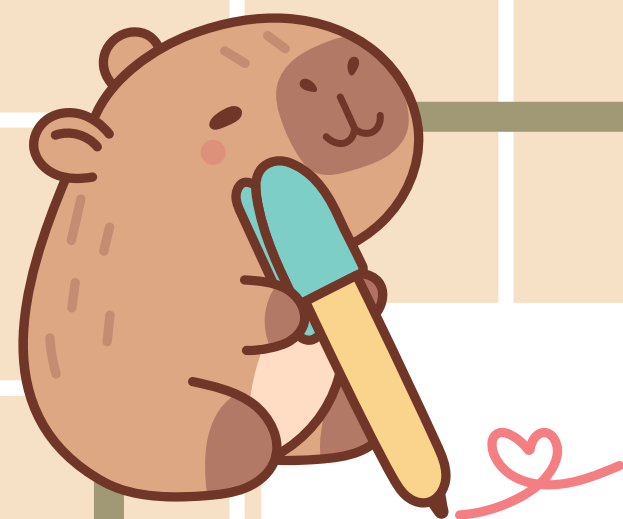

MODEL & PERSISTENCE Layer – Cloudinary Storage

```
// Fetch temporary binary data, convert to base64 WebP, and save to CDN
const response = await client.fetch(outputUrl);
const arrayBuffer = await response.arrayBuffer();
const buffer = Buffer.from(arrayBuffer);

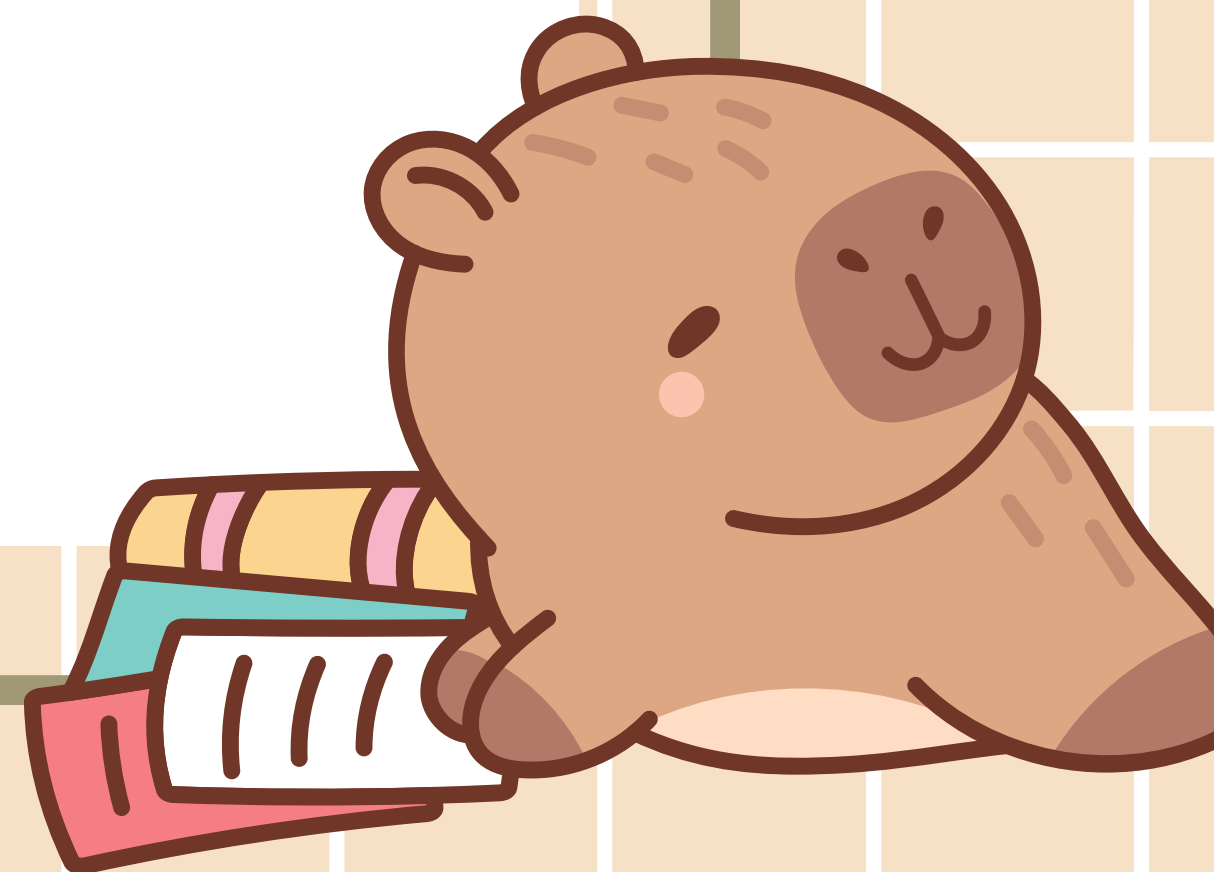
// Converting buffer to inline base64 WebP URI
const base64Image = `data:image/webp;base64,${buffer.toString("base64")}`;

// Saving the persistent asset state in Cloudinary
const uploadResult = await cloudinary.uploader.upload(base64Image, {
  folder: "tryon_results",
  resource_type: "image",
});

// Return the permanent URL to the Controller to reply to the View
const resultImageUrl = uploadResult.secure_url;
res.json({ success: true, resultImageUrl, clothType });
```



DEMO





Thank You

Any question ?